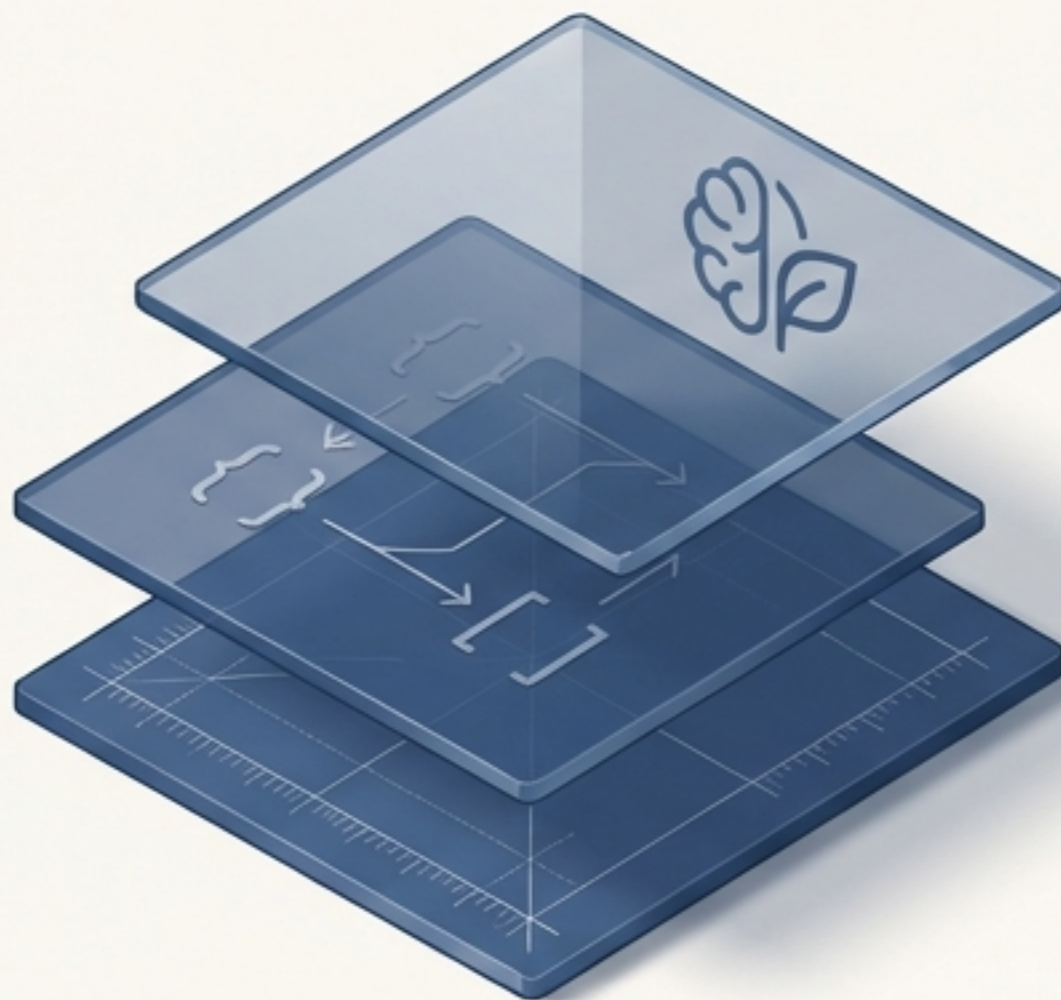


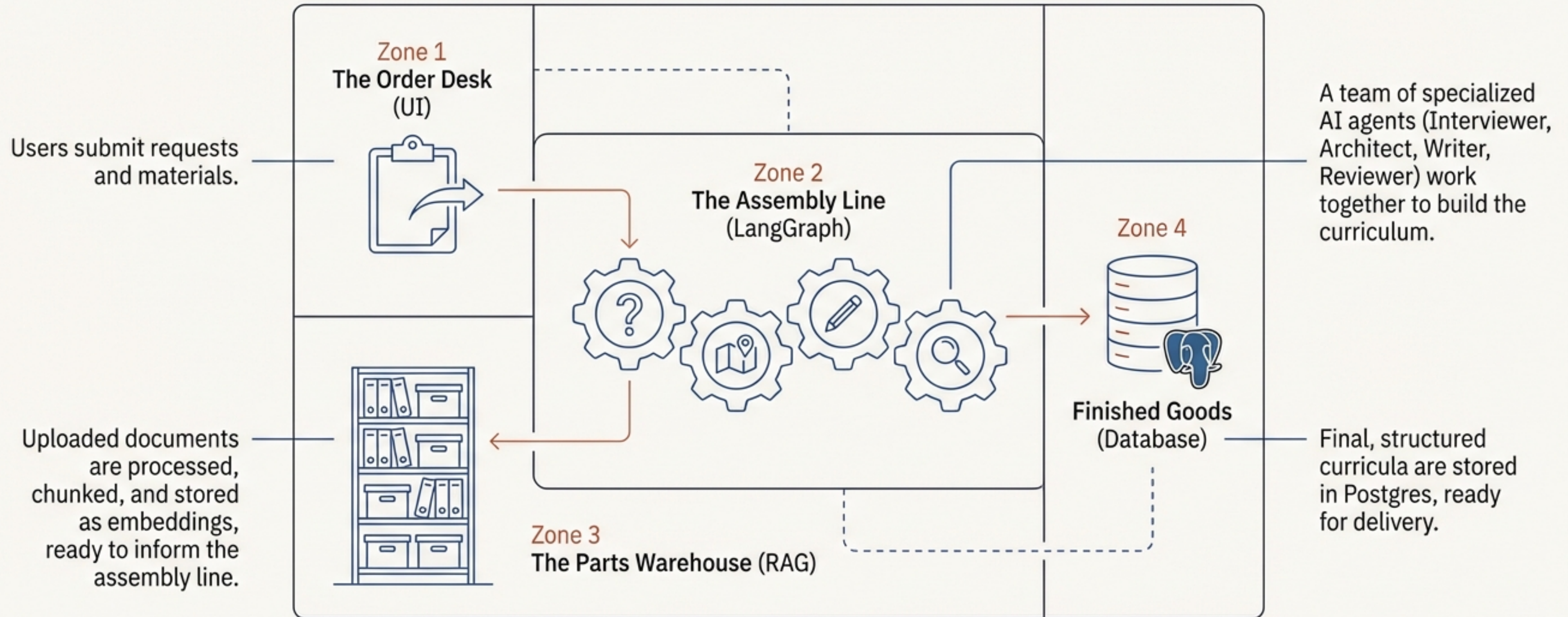
Lumina: An AI-Powered Learning Platform

A technical overview for developers.

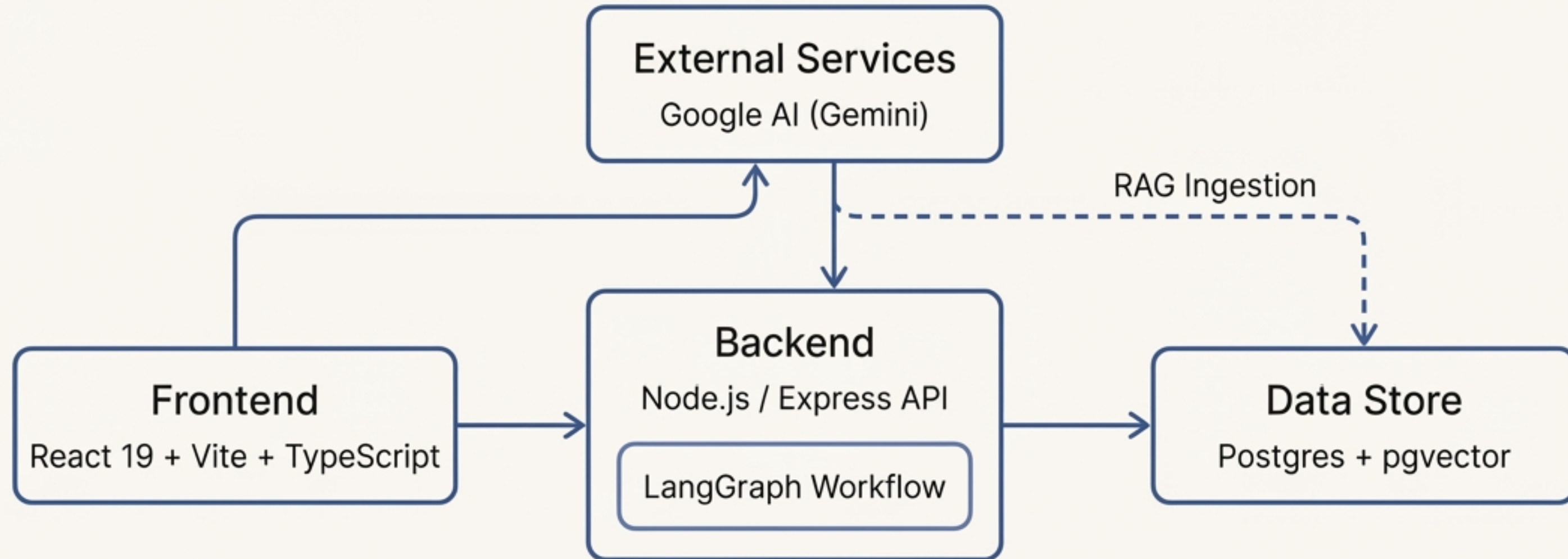


Lumina is an immersive learning platform built on a React/Vite SPA and a Node/Express API. It orchestrates multi-agent Gemini workflows to generate personality-aware courses with structured JSON output, optionally grounded by Retrieval-Augmented Generation (RAG).

Understanding Lumina as an AI Learning Factory



The Architectural Blueprint



Frontend

React 19, Vite,
TypeScript

Backend

Express (Node),
LangGraph

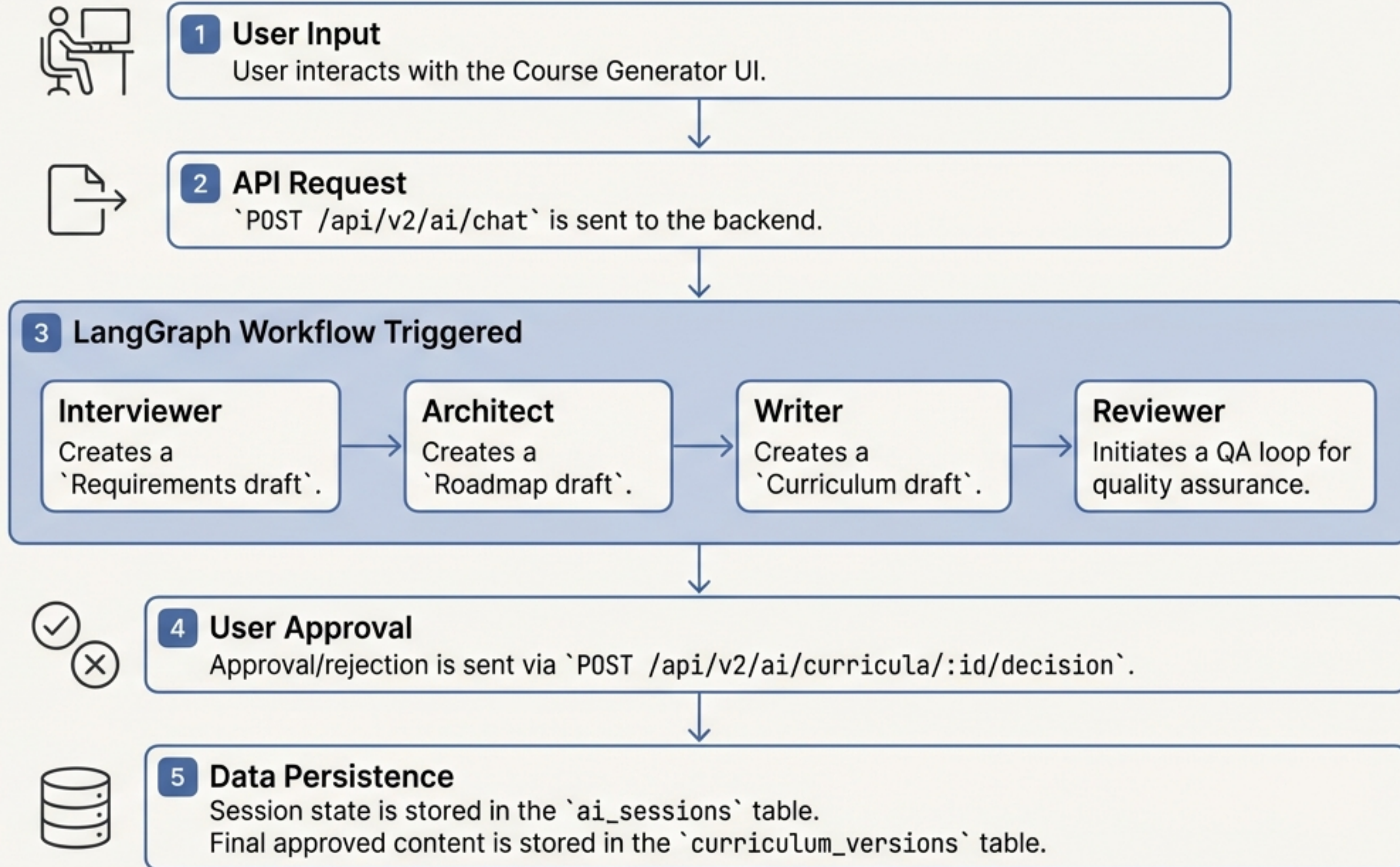
Data

Postgres, pgvector
(optional)

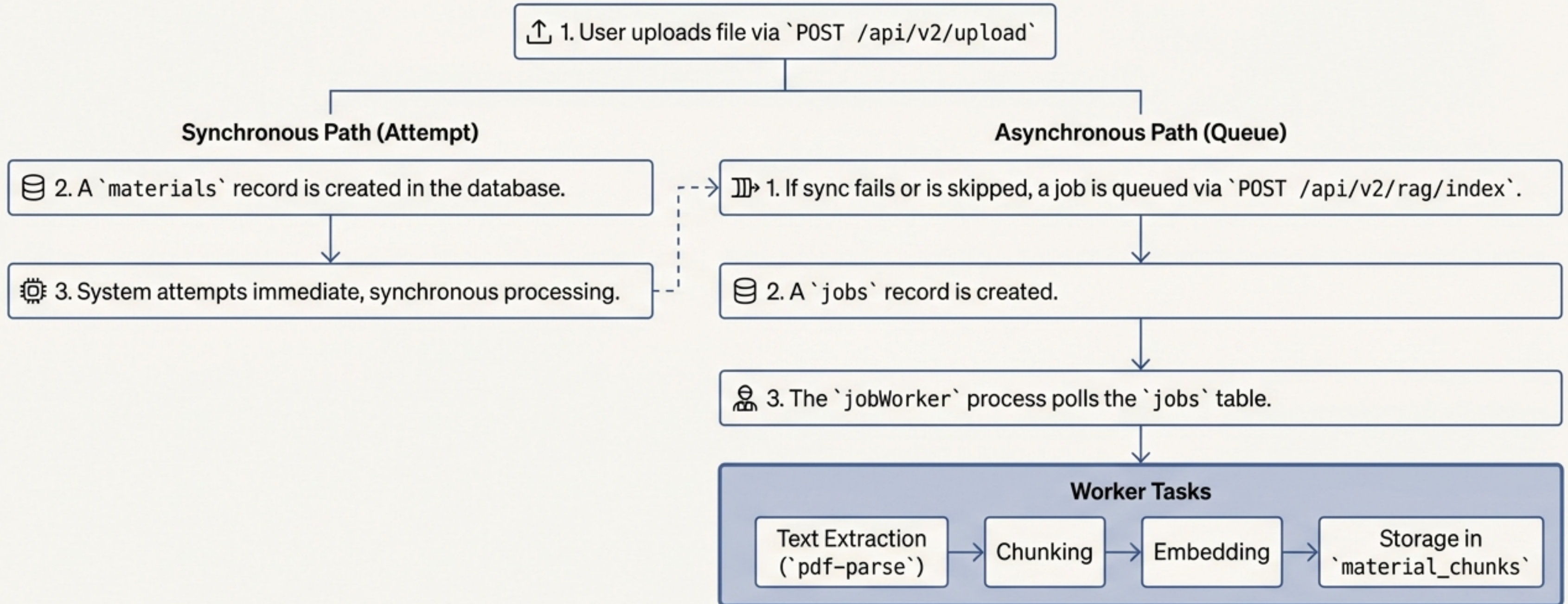
AI

Google Gemini
(`@google/genai`)

Flow 1: The AI Curriculum Generation Loop



Flow 2: RAG Ingestion for Grounding Content



Retrieval

Uses pgvector cosine distance for semantic search when available, with a simple text search fallback.

Bringing Content to Life: Frontend Rendering

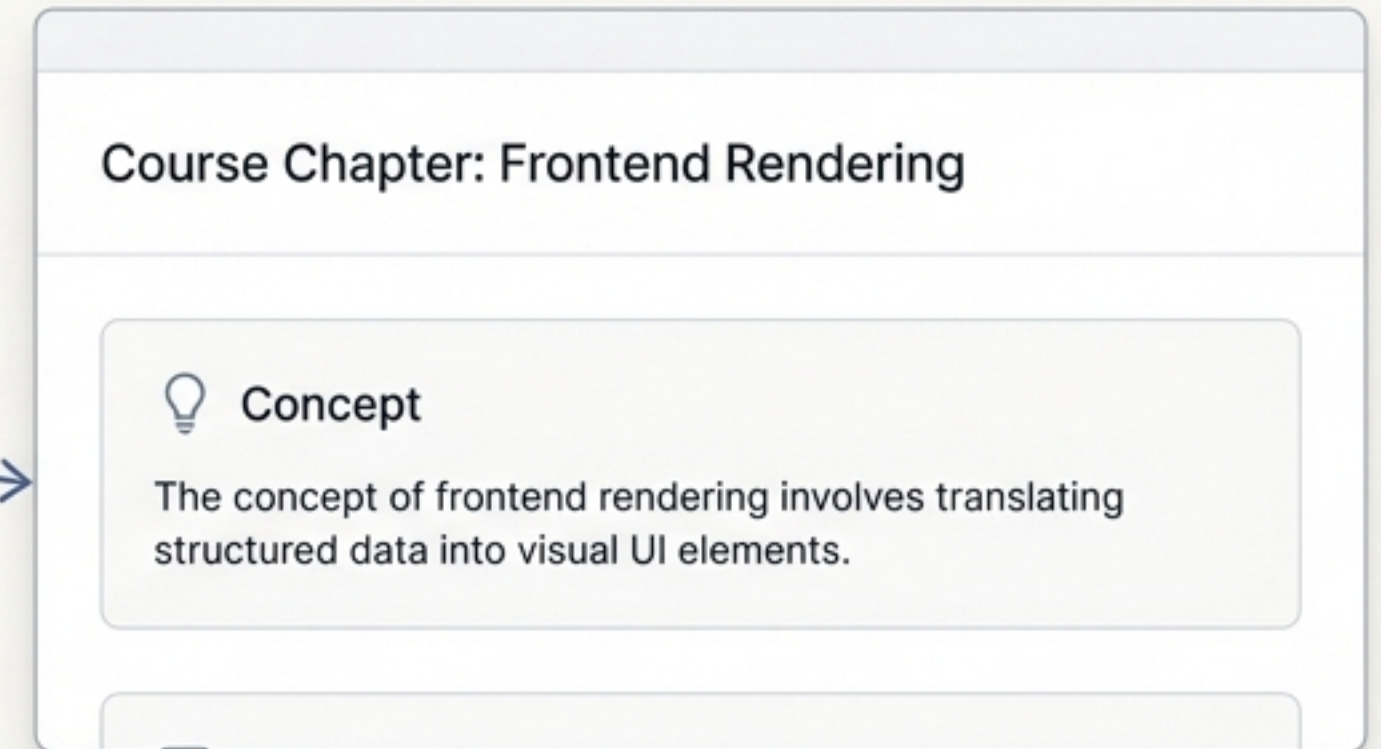
Structured Data Model

```
{
  "title": "Frontend Rendering",
  "chapters": [
    {
      "blocks": [
        {
          "type": "concept",
          "content": "The concept of frontend rendering involves translating structured data into visual UI elements."
        }
      ]
    }
  ]
}
```

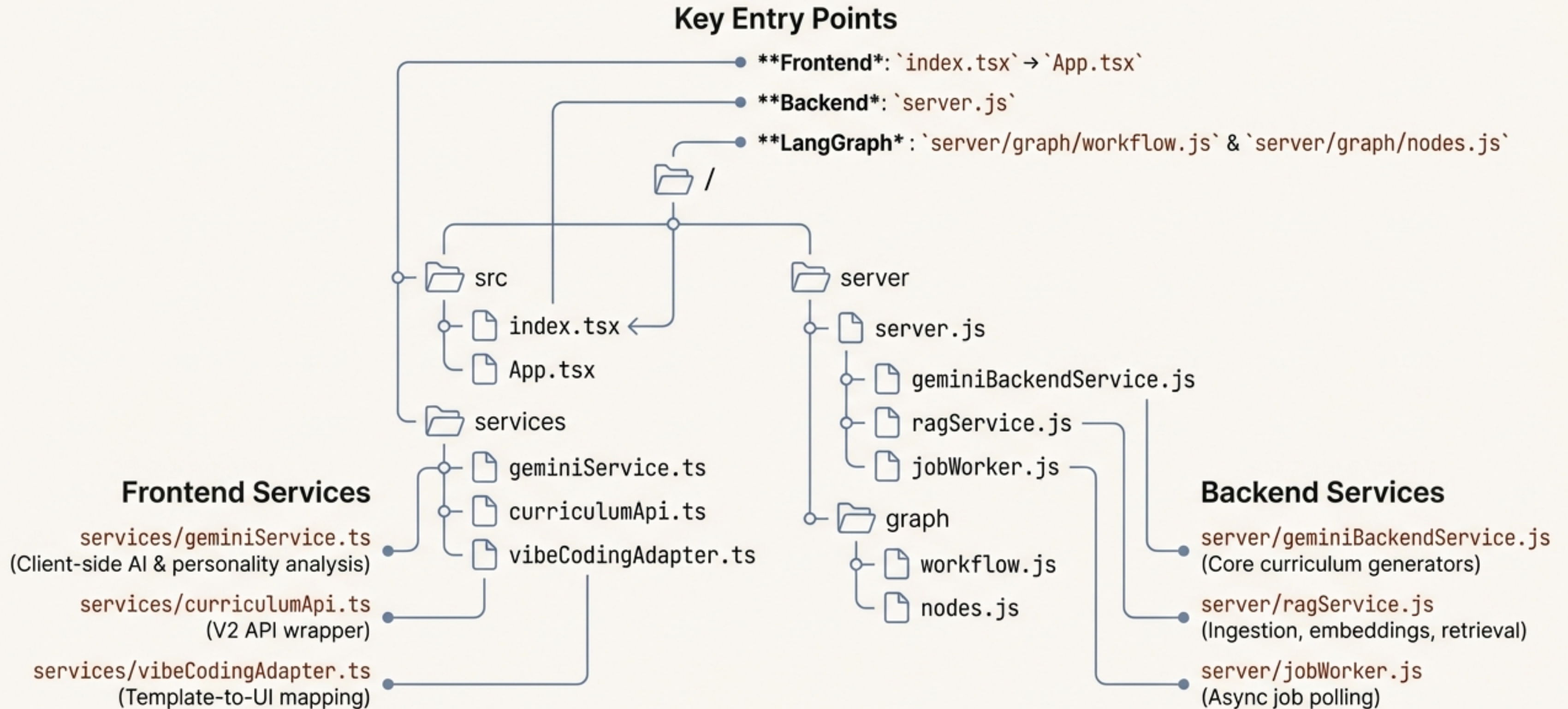
The `vibe\_coding` JSON template is the core structured storage format. The `vibeCodingAdapter` maps this data to UI components.

Supported Block Types

- 💡 Concept
- 💬 Dialogue
- 🛠 Workshop
- 📖 Reflection
- ☑ Checklist



Navigating the Codebase: A Repository Map



The Data Layer: Database and Static Assets



Dynamic Data (Postgres)

The primary data store for generated content, user sessions, and RAG materials.

Key Tables

- `ai_sessions`
- `curriculum_versions`
- `materials`
- `material_chunks`
- `jobs`

Note

`pgvector` extension is optional but recommended for semantic search capabilities.



Static Data (File System)

Pre-built curricula and reference materials checked directly into the repository.

Locations

Curricula

- `data/curricula/` (e.g., `vibe_coding`, `unity_ai`, `blender`)

Glossary

- `data/glossary/terms.ts`

Specs

- `doc/`

Getting Started: Local Development Setup

1. Prerequisites

-  Node.js (v20+)
-  PostgreSQL (pgvector extension is optional)
-  A valid Gemini API Key

2. Environment Variables

Create a `.env.local` file in the root directory.

```
GEMINI_API_KEY=your_key  
DATABASE_URL_PHASE1=postgres://user:pass@localhost:5432/lumina_db  
PHASE1_USER_ID=00000000-0000-0000-0000-000000000001
```

3. Database Setup

Apply the schema:

```
doc/ai-curriculum-spec/local_postgres_phase1.sql
```

(Optional) Seed data with `scripts/migrate_curricula.js` and `scripts/migrate_curricula.js` and `scripts/seed_full_integrated.js`.

4. Run the Application

```
npm install  
npm run dev
```

****Access URLs**:**

Frontend: `http://localhost:3007`

Backend: `http://localhost:3006`

The Lumina V2 API Endpoint Reference

All endpoints are prefixed with `/api/v2`. The primary API client is located at `services/curriculumApi.ts`.

AI Generation & Curriculum Management

POST `/ai/chat` Start a new generation workflow

POST `/ai/curricula/:id/decision` Approve/reject a draft

GET `/curricula` List all curricula

GET `/curricula/:id` Fetch a specific curriculum

RAG & Data Ingestion

POST `/upload` Upload a new material for RAG

POST `/rag/index` Queue an async ingestion job

Job Management

GET `/jobs/:id` Check the status of an async job

Roadmap & Opportunities for Contribution

Lumina is an actively evolving project. Here are key areas where you can make an impact.



Build Native TTS Integration: Implement text-to-speech and voice selection features.



Automate Content Ingestion: Create a robust pipeline for Blender HTML and other formats.



Enhance AI Interaction: Develop NLP-based approval detection to move beyond simple button clicks.



Integrate Developer Tooling: Wire the MCP (tooling environment) into the runtime product flow.

Your contributions can help shape the next generation of AI-powered learning.